INTERNAL TECHNICAL REFERENCE

Narration Studio Architecture and Internal Workings

End-to-end documentation of the Microsoft Agent Framework boilerplate that turns a voice transcript into a synced narration video. Written from the actual source in `E:\exercises\maf`.

Solution NarrationStudio.slnx

Project src/NarrationStudio/NarrationStudio.csproj · net10.0-windows

Audience Developers onboarding to the codebase

Classification Internal. Contains architecture and configuration details; API keys are not reproduced here.

Date 21 August 2026

Status Confirmed against current source. Where behavior is implied by a library rather than this repo, that is labelled *inferred*.

Document control

Item	Value
Title	Narration Studio — Internal Technical Architecture and Workings
Scope	Entire application in this repository. There is a single executable project and no other services.
Source of truth	C# source under `src/NarrationStudio`, `appsettings.json`, `README.md`, and observed runtime behavior.
Out of scope	Microsoft Agent Framework library internals except as used here. ElevenLabs / Ollama / ffmpeg internals except as called.
Secrets policy	This document names configuration keys but does not print secret values.

> **HOW TO READ "CONFIRMED" VS "INFERRED"**
> **Confirmed** means the statement is directly visible in this repository's source or configuration. **Inferred** means the behavior comes from a NuGet package, OS API, or external service that this repo calls but does not implement. Those inferences are called out so a new developer does not treat library magic as local code.

Table of contents

1. **Application Overview**
2. **System Architecture**
3. **Detailed Component Breakdown**
4. **Request/Response and Execution Flows**
5. **Database and Data Layer**
6. **API Documentation**
7. **Frontend / Internal UI Workings**
8. **Backend / Internal Service Workings**
9. **Security**
10. **Error Handling and Edge Cases**
11. **Configuration and Environment**
12. **Deployment and Infrastructure**
13. **Performance and Scalability**
14. **Testing**

15. Developer Guide

16. Complete Dependency Map

17. Technical Diagrams

18. Appendix

1. Application Overview

1.1 Purpose and core functionality

Narration Studio is a **Windows console application** that produces a timestamped narration video from a source transcript. It is a Microsoft Agent Framework (MAF) boilerplate: specialist agents can drive the same production tools that a deterministic runner also calls directly.

The product contract, as implemented, is:

1. Read `voice_transcript.txt` (raw notes / transcript).
2. Rewrite it into a spoken ~7-minute talk, saved as `output/scripts/narration-rewritten.txt`. Today this rewrite is usually **stubbed** from `voice_transcript_ollama.txt` when `Studio:UseStubNarration` is true; otherwise it calls local Ollama.
3. Split that rewritten talk into exactly **8** narration scripts.
4. Plan at most **5** on-screen phrases per script and render them as PNG slides.
5. Synthesize `audio-01.mp3` … `audio-08.mp3` via ElevenLabs (if an API key is configured) or Windows SAPI + ffmpeg.
6. Sync each MP3 with that segment's slides into `video-01.mp4` … `video-08.mp4`.
7. Merge into `video_yyyyMMdd_HHmmss.mp4` in the output folder.

The application is **not** a web service, desktop GUI, or multi-user system. There is no ASP.NET host, no HTTP listener, no SPA, and no SQL database. A developer expecting MVC / REST layers will not find them; the analogue of "controllers" is `ProductionApp` plus tool methods on `ProductionTools`.

1.2 Main use cases

Use case	How it is invoked	What happens
Produce a video from the repo transcript (current default)	`dotnet run --project src\NarrationStudio -- --orchestration direct` from the repo root	Clears previous output, copies stub narration, segments, slides, TTS, compose, merge.
Agent-orchestrated production	Omit `--orchestration` (defaults to `handoff`) or pass `sequential`	Builds MAF agents against Ollama/OpenAI/Azure; specialists call the same tools. If the merged file is missing afterward, DirectPipelineRunner finishes the job.
Use a different transcript file	Pass a path as the first non-flag argument, or set `Studio:TranscriptPath`	`ProductionApp.ResolveTranscriptPath` walks a candidate list until a file exists.
Call Ollama for the rewrite	Set `Studio:UseStubNarration` to `false`	`NarrationRewriteService.RephraseAsync` posts the raw transcript to Ollama.

instead of the stub		
Prevent overlapping runs	Automatic	Named mutex `NarrationStudio.Production`. A second instance throws.

1.3 High-level architecture

Composition is Generic Host (`Host.CreateApplicationBuilder` in `Program.cs`). Every collaboration object is a singleton. Control flows top-down: host → `ProductionApp.RunAsync` → either `WorkflowHost` (agents) or `DirectPipelineRunner` → `ProductionTools` → domain services → disk and external processes/APIs.

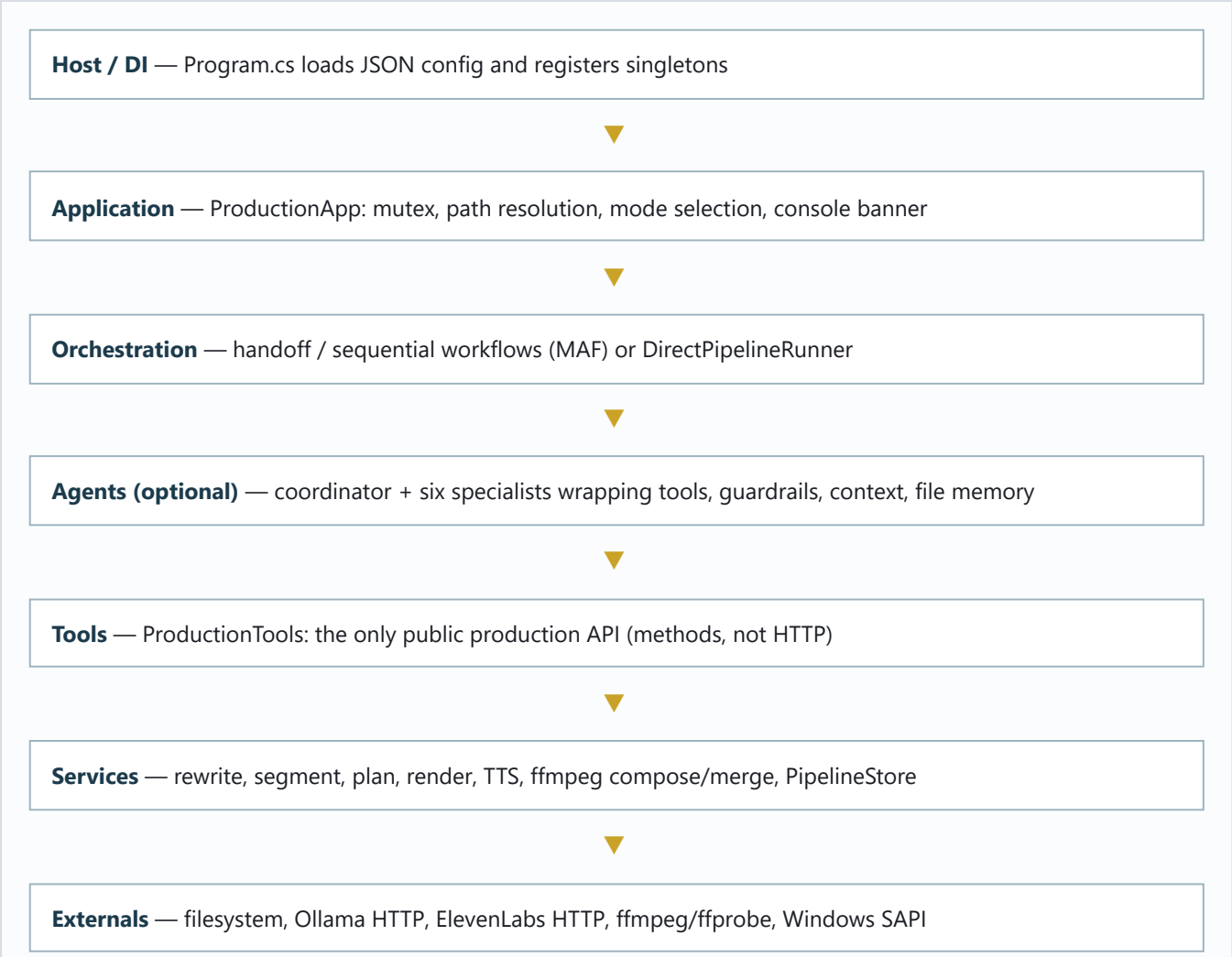


Figure 1. Logical layers. Confirmed: all boxes correspond to types or processes named in this repo.

1.4 Overall data and control flow

Data is a single in-memory `PipelineState` cloned on read via JSON round-trip, persisted after every mutation to `output/pipeline-state.json`. Artifacts are files. There is no message bus.

Control flow for a typical current run (`UseStubNarration=true`, `--orchestration direct`):

1. `Program.cs` builds the host and calls `ProductionApp.RunAsync(args)`.
2. Mutex acquired. Transcript and output directory resolved. `PipelineStore.Initialize` deletes previous generated files.
3. Because stubbing is on, `RephraseTranscriptIntoNarration` copies the stub file into `narration-rewritten.txt` and sets `NarrationPath`.
4. Direct mode always re-segments, re-plans, re-renders, re-synthesizes, re-composes, and merges.
5. Console prints `Done: {merged path}`.

Status strings written onto state: `created` → `narration-ready` → `scripts-ready` → `slides-planned` → `slides-rendered` → `audio-ready` → `videos-ready` → `complete`.

2. System Architecture

2.1 Frontend, backend, database, APIs, services, external dependencies

Map the requested web-app vocabulary onto what actually exists:

Typical web layer	Narration Studio equivalent	Confirmed location
Frontend	Console stdout/stderr. No UI framework, routing, or components.	`ProductionApp`, `WorkflowHost.RunAsync`, `DirectPipelineRunner`
Backend	The same process. Business logic is C# services invoked in-process.	`src/NarrationStudio/Services`, `Tools`, `Orchestration`
Database	None. Persistence is JSON + files under `output/`.	`PipelineStore`, `Models/PipelineModels.cs`
HTTP API (this app)	None served. The process is a client of Ollama and ElevenLabs.	`NarrationRewriteService`, `TtsService`
Internal API	`ProductionTools` methods exposed to agents via `AIFunctionFactory.Create`	`Tools/ProductionTools.cs`
External processes	`ffmpeg`, `ffprobe` on PATH	`FfmpegProcess` in `TtsService.cs`

2.2 Responsibilities of each layer

Layer	Responsibility	Must not
Program.cs	Configuration providers, DI, start `ProductionApp`	Contain pipeline logic

ProductionApp	Single-run policy, path/mode resolution, stub rewrite before agents, failover to direct	Segment text or call ffmpeg
WorkflowHost / AgentFactory	Build MAF agents and workflows; stream agent text to the console	Write media files themselves
DirectPipelineRunner	Deterministic ordered tool calls; overwrite-oriented regeneration	Talk to LLMs
ProductionTools	Validate preconditions, call services, mutate PipelineStore, return short status strings	Know ffmpeg flags
Domain services	Pure-ish transforms and I/O (rewrite, split, PNG, TTS, video)	Choose orchestration mode
PipelineStore	Thread-safe state, filenames, wipe-on-start, persist JSON, agent status summary	Call external APIs
Guardrails / context / file memory	Filter agent text, inject status into prompts, optional on-disk agent notes	Drive the media pipeline

2.3 How the layers communicate

All communication is **in-process method calls** through constructor-injected singletons. Agents do not get a private copy of services; they receive `AIFunction` wrappers around the same `ProductionTools` instance. That is why a tool called by an agent updates the same `PipelineStore` that DirectPipelineRunner later reads.

External communication:

- **Ollama** — HTTP via `OllamaApiClient` (OllamaSharp) at `Model:Endpoint`, default `http://localhost:11434`.
- **ElevenLabs** — HTTP POST `https://api.elevenlabs.io/v1/text-to-speech/{voiceId}` with header `xi-api-key`.
- **ffmpeg / ffprobe** — child processes, stdout/stderr captured asynchronously to avoid deadlock, 8-minute timeout.
- **Windows SAPI** — `System.Speech.Synthesis.SpeechSynthesizer` writing WAV, then ffmpeg to MP3.

2.4 Why this shape

- **Tools as the real pipeline.** Agents are optional drivers. Direct mode exists so production still works when the LLM is slow, missing, or poorly follows handoff instructions. Confirmed in `ProductionApp`: agent failure to produce a merged file always falls through to `DirectPipelineRunner`.
- **Wipe outputs on start.** Earlier versions skipped existing PNGs/MP3s/MP4s, so a new narration could be muxed with old media. `Initialize` now deletes generated trees.
- **Stub rewrite.** Local Ollama rewrite is long-running (timeout 30 minutes). Stubbing lets the rest of the factory be developed against a known talk.
- **Windows TFM.** `net10.0-windows` is required for `System.Speech`.

3. Detailed Component Breakdown

This section covers every significant type in the project. Small helpers are included because they affect production reliability.

3.1 Program.cs — composition root

Purpose. Create Generic Host, bind options, register singletons, run the app. There is no `Main` class; top-level statements are used.

Config loading (confirmed, and easy to get wrong). `Host.CreateApplicationBuilder` uses the process current directory as content root. Launch profile `workingDirectory` is the repo root (`$(ProjectDir)..\..`), so the first `appsettings.json` lookup is `{cwd}/appsettings.json` , which does **not** exist. The second explicit call loads `AppContext.BaseDirectory/appsettings.json` — the copy MSBuild places next to the DLL. That is the file that actually configures a `dotnet run` from the repo root.

Inputs / outputs. Input: `args` , JSON files, environment variables (Host default providers). Output: constructed `IServiceProvider` , then `ProductionApp.RunAsync` .

Error handling. None locally; failures bubble from `RunAsync` and crash the process (non-zero exit). Confirmed: no try/catch in `Program.cs` .

3.2 ProductionApp

File: `src/NarrationStudio/ProductionApp.cs`

Responsibilities. Exclusive run (mutex), resolve transcript/output/orchestration, initialize store (which wipes artifacts), print a banner (including TTS provider), optionally apply stub rewrite **before** agents exist, choose direct vs agent workflow, failover to direct if merge is missing, print Done.

Important methods.

Method	Behavior
`RunAsync(string[] args)`	Full run lifecycle described in §4.
`ResolveTranscriptPath`	Candidates: first non- `--` arg, then `Studio:TranscriptPath` , then several relative `voice_transcript.txt` locations. First existing file wins. Else `FileNotFoundException` .
`ResolveOutputDirectory`	`Studio:OutputDirectory` or `output` , rooted against current directory.
`ResolveOrchestration`	`--orchestration=x` or `--orchestration x` . Default `handoff` .

Design decision — stub before TryCreate. Creating agents calls `OllamaChatClientFactory.Create` and logs Ollama immediately. If stub rewrite waited until a rewriter tool ran, users saw Ollama connecting and assumed the stub was ignored. Stub copy now happens first.

Auth. None. Process identity is the logged-in Windows user.

3.3 PipelineStore and PipelineState

Files: `Services/PipelineStore.cs`, `Models/PipelineModels.cs`

Purpose. Single source of pipeline truth for one process run. Analogous to a unit-of-work + repository, but for files.

Internal structure. Private `PipelineState _state` plus `object _gate`. `Snapshot()` serializes then deserializes with source-generated `PipelineJsonContext` so callers cannot mutate live state without `Mutate`. Every `Mutate` writes `output/pipeline-state.json`.

Initialize. Sets transcript and output paths, creates directories, **deletes** previous scripts/slides/audio/video, `agent-file-memory`, leftover `video_*.mp4`, and old state JSON, then persists a fresh state. Returns the count of deleted files for the banner.

Naming helpers (confirmed).

```
audio-{nn}.mp3
video-{nn}.mp4
segment-{nn}-slide-{nn}.png
narration-rewritten.txt
video_{RunId}.mp4   where RunId = yyyyMMdd_HHmmss at store construction
```

State fields. See §5. `RunId` is assigned in the constructor **before** Initialize; wiping old merged videos uses glob `video_*.mp4`, not the new RunId.

SummarizeForAgent. Counts scripts/slides/images/audio/videos from in-memory segments plus File.Exists. "Next required step" is a priority chain. After wipe, in-memory segments are empty even if a human dropped files on disk mid-run — agents are steered by this summary, not by scanning leftover files (except existence checks on paths already recorded in state).

Concurrency. Lock around snapshot/mutate/initialize. Tool methods sometimes snapshot, work, then mutate — a second thread could interleave, but the app is effectively single-threaded aside from ffmpeg async reads.

Why JSON clone for Snapshot. Cheap isolation without implementing `ICloneable` on a graph of lists. Cost is negligible at this scale.

3.4 ProductionTools

File: `Tools/ProductionTools.cs`

Purpose. The internal production API. Each public method has a `[Description]` consumed by MAF when wrapping as an `AIFunction`. Direct mode calls the same methods.

Methods, inputs, outputs.

Method	Input	Writes	State status	Throws if
`GetPipelineStatus`	—	—	—	—
`LoadTranscript`	—	—	—	file missing
`LoadNarrationSource`	—	—	—	no NarrationPath file
`RephraseTranscriptIntoNarration`	raw transcript via LoadTranscript; rewrite service may ignore it when stubbing	`scripts/narration-rewritten.txt`	narration-ready	stub missing/short; Ollama empty
`SegmentTranscriptIntoEightScripts`	rewritten text	`narration-01..08.txt` , `narration-all.txt`	scripts-ready	count ≠ SegmentCount
`SaveNarrationScripts`	JSON array of segments	same script files	scripts-ready	parse fail; wrong count
`PlanSlidesForAllSegments`	each segment.Script	in-state SlideSpec lists	slides-planned	scripts missing
`RenderAllSlideImages`	slide phrases	PNGs; always overwrite	slides-rendered	scripts missing
`SynthesizeAllAudioAsync`	each script	MP3s; duration probed	audio-ready	scripts missing; TTS/ffmpeg fail
`ComposeAllSegmentVideos`	audio + images	segment MP4s	videos-ready	audio/slides missing
`MergeAllVideos`	8 video paths	merged MP4	complete	count ≠ SegmentCount

EnsureScripts. Requires `Segments.Count == SegmentCount` in memory — not a directory listing. After a wipe, agents that skip segmentation cannot render.

Cancellation. `Rephrase` and TTS pass `CancellationToken.None` today. Host stop (`ApplicationStopping`) is only wired into `WorkflowHost.RunAsync` , not into ffmpeg/TTS. Confirmed limitation.

3.5 NarrationRewriteService and OllamaChatClientFactory

File: `Services/NarrationRewriteService.cs` (both types)

OllamaChatClientFactory. Lazy singleton `IChatClient` (`OllamaApiClient`) with a .NET `Lock` . HttpClient BaseAddress = endpoint, Timeout = max(1, TimeoutMinutes) minutes. Used both for rewrite and, when no Azure/OpenAI key, for agent orchestration.

RephraseAsync.

- If `UseStubNarration` : resolve stub path (transcript dir, cwd, BaseDirectory, and a walk-up from bin). Strip `<think>` and fences. Reject if length < 400 characters. Does not send the raw transcript to a model.
- Else: system prompt asks for ~ `TargetTalkMinutes * 140` words, spoken voice, no invented facts. User message is the raw transcript. Temperature 0.35. Same length check on the result.

Why 400 characters. Heuristic to catch empty or "thinking only" model output. Not a linguistic quality gate.

3.6 TranscriptSegmenter

Splits on `(?<=[\.!\?])\s+` , then packs sentences into `SegmentCount` buckets by character budget. If too few groups, it peels sentences from the richest bucket. Titles are "Segment NN — {first sentence truncated to 72 chars}".

WriteScriptFiles overwrites `narration-{nn}.txt` and concatenates `narration-all.txt` .

Edge case (confirmed). If every remaining group has a single sentence and count is still below 8, the loop can `break` after duplicating one sentence into a new group, then `Take(buckets)` may still return fewer than 8. `SegmentTranscriptIntoEightScripts` would then throw. Unlikely for a 7-minute talk; possible for a tiny stub.

3.7 SlidePlannerService

Clamps max slides to 1..5. Sentence-splits the script; if more sentences than max, concatenates chunks. Phrases truncated to 220 characters with ellipsis. Returns `List<SlideSpec>` with numbers and phrases; `ImagePath` filled later by render.

This is deterministic NLP-by-regex, not an LLM. The `slide_planner` agent is instructed to call this tool rather than invent phrases, though a misbehaving model could still skip the tool (handoff risk). Direct mode always calls the tool.

3.8 SlideImageRenderer

ImageSharp PNG, 1920×1080 by default. Dark navy canvas, gold rule, header "SEGMENT nn · SLIDE x/y", wrapped body text, footer = segment title. Fonts tried: Segoe UI, Arial, Calibri, else first system family.

WriteAllBytesWithRetry. FileMode.Create with share ReadWrite|Delete, 6 attempts, then write a GUID sidecar and copy over. Exists because Windows file locks from ImageSharp, explorers, or leftover ffmpeg were observed during development.

3.9 TtsService and FfmpegProcess

Key resolution. `Studio:ElevenLabsApiKey` first, else env `ELEVENLABS_API_KEY`. If present: POST MPEG to ElevenLabs; on non-success throw with status and body (no fallback to SAPI). If absent: SAPI WAV, or NAudio sine "tone bed" if SAPI throws, then ffmpeg libmp3lame.

DescribeProvider is used in the startup banner so operators can see ElevenLabs vs SAPI without reading logs.

FfmpegProcess. Starts ffmpeg/ffprobe with redirected stdout and stderr **both read asynchronously**, WaitForExit 8 minutes, kill process tree on timeout. This avoids the classic deadlock of reading one stream to end before the other. Probe returns stdout; run returns stderr (unused except on failure).

Duration. ffprobe format duration; parse fail defaults to 5 seconds (would distort slide timing if probe fails).

3.10 VideoComposerService

ComposeSegment. Writes ffmpeg concat demuxer list: each slide `file` + `duration` = max(0.4, audioDuration / slideCount), last file repeated (ffmpeg concat still-image requirement). Mixes concat video with the MP3, scale/pad to configured size, fps, libx264 stillimage, aac, `-shortest`, faststart. `-y` overwrites.

Merge. Concat list of the eight MP4s. Tries stream copy first; on any exception re-encodes libx264+aac. That fallback is why mismatched timebases from vfr concat can still produce a playable master.

Path escaping for concat files: backslash → slash, single quote → ffmpeg concat escape.

3.11 DirectPipelineRunner

Logs, prints status, rephrases only if `NarrationPath` is empty/missing (so ProductionApp's stub is not duplicated unless something deleted the file), then **always** segments, plans, renders, synthesizes, composes, merges. No skip-if-exists on media. Return value is the merge tool's string; ProductionApp also reads `MergedVideoPath` from state.

3.12 AgentFactory and ProductionCrew

ProductionCrew is a record of seven `AIAgent` instances.

TryCreate. Builds one `IChatClient`: Azure OpenAI if endpoint set (key or DefaultAzureCredential), else OpenAI if `OPENAI_API_KEY`, else Ollama. Failure returns false → ProductionApp uses direct mode.

Each agent: `chatClient.AsAIAgent(ChatClientAgentOptions)` with id/name/description, instructions, tools, and two context providers. Then `.AsBuilder().Use(guardrails.RunAsync).Use(guardrails.FunctionAsync).Build()`.

File memory: MAF `FileMemoryProvider` + `FileSystemAgentFileStore` under `output/agent-file-memory`, working folder `production-notes`. **Inferred:** the package persists agent notes as files; this repo does not reimplement that store. Initialize deletes that directory each run, so memory does not survive reruns.

Chat client priority is Azure → OpenAI → Ollama. A developer with Azure env vars set will **not** use local Ollama for specialists even if Ollama is running. Confirmed in `CreateChatClient`.

3.13 WorkflowHost

Handoff. `AgentWorkflowBuilder.CreateHandoffBuilderWith(coordinator)`. If stubbing, rewriter is omitted from the graph. Specialists hand back to coordinator. Coordinator has autonomous mode, turnLimit 8, continuation prompt to GetPipelineStatus. **Inferred:** MAF runtime interprets handoff conditions as natural-language routing hints to the model, not compiled predicates.

Sequential. `BuildSequential` with rewriter omitted when stubbing.

RunAsync. `InProcessExecution.RunStreamingAsync`, sends `TurnToken(emitEvents: true)`, prints `AgentResponseUpdateEvent` text grouped by executor id, returns on `WorkflowOutputEvent`.

BuildUserPrompt. Injects stub vs Ollama rewrite step plus live `SummarizeForAgent`.

3.14 ProductionGuardrails

Inbound user messages containing blocked terms (`harmful`, `illegal`, `violence`, `exploit`, `malware`, `bypass paywall`) are replaced with a blocked marker; the run short-circuits with a refusal. All messages get email and `###-###-####` phone redaction.

FunctionAsync logs tool start/finish. It counts the word “slide” in string results of tools whose names contain “Slide” and logs a warning if count exceeds max slides × segments. **Confirmed: it does not trim or reject the result.** The 8-script cap is enforced in tools by throwing, not here.

Guardrails apply only to agent runs, not DirectPipelineRunner.

3.15 ProductionContextProvider

On invoke, injects instructions: current summary, exact file naming, segment/slide caps, stay on task. After invoke, appends truncated assistant text to `PipelineState.Notes` (max 24, drop oldest). Notes are persisted in pipeline-state.json.

3.16 Configuration types

`StudioOptions` section `Studio`; `OllamaOptions` section `Model`. Defaults in the class match appsettings for segment counts, stubbing, voice id, video size. Options are captured at construction (`IOptions<T>.Value`), so `reloadOnChange` on JSON files does **not** update live option objects already injected. Confirmed: services store `_options = options.Value`.

3.17 Launch profile

`Properties/launchSettings.json` profile `NarrationStudio`: empty args, working directory repo root. That is why transcripts at repo root are found and output is `E:\exercises\maf\output` rather than under `bin`.

4. Request/Response and Execution Flows

There are no inbound HTTP requests. “Request” here means a developer invoking the executable.

4.1 Direct orchestration (confirmed current happy path)

```
User: dotnet run --project src\NarrationStudio -- --orchestration direct
  → Host.CreateApplicationBuilder
  → Bind Studio + Model options
  → ProductionApp.RunAsync
      → Mutex NarrationStudio.Production
      → Resolve voice_transcript.txt (repo root)
      → Resolve output/ (cwd-relative)
      → orchestration = "direct"
      → PipelineStore.Initialize  (delete previous artifacts)
      → Banner: Transcript, Output, Mode, Previous cleared N, Rewrite stub, TTS ElevenLabs|SAPI
      → UseStubNarration → ProductionTools.RephraseTranscriptIntoNarration
            → LoadTranscript (raw file, unused by stub rewrite)
            → NarrationRewriteService.RephraseAsync → read stub, strip think tags
            → write narration-rewritten.txt, status=narration-ready
      → DirectPipelineRunner.RunAsync
            → skip rephrase (NarrationPath exists)
            → SegmentTranscriptIntoEightScripts
            → PlanSlidesForAllSegments
            → RenderAllSlideImages
            → SynthesizeAllAudioAsync  (8 sequential ElevenLabs or SAPI calls)
            → ComposeAllSegmentVideos  (8 ffmpeg)
            → MergeAllVideos           (ffmpeg concat)
      → Console: Done: output\video_{RunId}.mp4
```

4.2 Handoff orchestration

1. Same init and optional stub rewrite.
2. `AgentFactory.TryCreate` — if this throws/returns false, jump to direct.
3. `CreateHandoffWorkflow` then `RunAsync` with user prompt and `ApplicationStopping`.
4. Coordinator should call `GetPipelineStatus` and hand off. Specialists call tools that mutate store and disk.
5. Autonomous continuation up to 8 coordinator turns *(inferred MAF semantics)*.
6. If `MergedVideoPath` missing or file absent → warning log → DirectPipelineRunner. Direct will not skip media generation, so a partial agent run is overwritten into a complete video (scripts already in memory if agents segmented; if not, direct segments from rewritten file).

Subtlety. Direct after a partial agent run still requires in-memory `Segments` or it will call `SegmentTranscriptIntoEightScripts` from the rewritten file. Agent-authored scripts in memory are kept for that process; they are not reloaded from disk on a new process because Initialize wipes and does not hydrate from leftover JSON (JSON was also deleted).

4.3 Sequential orchestration

Same as handoff but `BuildSequential` runs specialists in a fixed list. Stubbing removes the rewriter from that list because ProductionApp already wrote the talk.

4.4 ElevenLabs request/response (external)

Item	Value (confirmed in TtsService)
Method / URL	POST https://api.elevenlabs.io/v1/text-to-speech/{ElevenLabsVoiceId}
Headers	`xi-api-key`, `Accept: audio/mpeg`
JSON body	`{ "text": "<segment script>", "model_id": "<ElevenLabsModelId>" }`
Success	Body copied to audio-NN.mp3
Failure	Exception including HTTP status and response body; pipeline stops; no SAPI fallback

4.5 Ollama rewrite request (when stubbing off)

Via Microsoft.Extensions.AI `GetResponseAsync` on `OllamaApiClient`. **Inferred wire format:** OllamaSharp talks to Ollama's HTTP API (typically `/api/chat`). This repo does not construct that JSON manually. Timeout is the HttpClient timeout (default 30 minutes).

4.6 Frontend-to-backend-to-database

Does not apply. Console is the UI; services are in-process; "database" is `pipeline-state.json` written on Mutate.

5. Database and Data Layer

There is **no database server, ORM, or migration framework**. Documented here is the file data layer that plays the same role.

5.1 Architecture

One JSON document plus a directory tree. Schema is the C# types in `PipelineModels.cs`, serialized with `System.Text.Json` source generation, camelCase, indented.

5.2 “Tables” (files and collections)

Store	Purpose	Lifetime
`output/pipeline-state.json`	Run metadata, segments, notes, paths	Rewritten every Mutate; deleted at next Initialize
`output/scripts/*`	Rewritten talk and 8 scripts	Deleted at next run start
`output/slides/*.png`	Slide images	Deleted at next run start
`output/audio/*.mp3`	TTS	Deleted at next run start
`output/video/*.mp4` and concat txt	Segment videos + ffmpeg lists	Deleted at next run start
`output/video_*.mp4`	Merged masters	All matching files deleted at next run start
`output/agent-file-memory/`	MAF file memory	Cleared at next run start

5.3 Relationships

PipelineState 1—* NarrationSegment 1—* SlideSpec. Paths on segments/slides are denormalized strings, not foreign keys. Integrity is “file exists at path” checks in tools and SummarizeForAgent.

5.4 Schema (important fields)

Type	Fields
PipelineState	runId, transcriptPath, narrationPath, outputDirectory, startedAt, status, segments[], mergedVideoPath, notes[]
NarrationSegment	number, title, script, slides[], audioPath, videoPath, audioDurationSeconds
SlideSpec	number, phrase, imagePath

5.5 Queries and access

No query language. Access is LINQ over in-memory lists inside locked Mutate/Snapshot. There is no index. Hydration from a previous JSON file is **not implemented**; Initialize always starts empty segments after wipe.

5.6 Validation and integrity

- Segment count must equal `Studio:SegmentCount` (default 8) at segment and merge boundaries.

- Slide lists trimmed to MaxSlidesPerSegment at render time.
- Stub/Ollama narration must be ≥ 400 characters.
- JSON save of agent scripts uses `PipelineJsonContext.ListNarrationSegment`.
- No checksums, no transactional disk writes (JSON write is not atomic replace; a crash mid-write can corrupt pipeline-state.json). Next run deletes it anyway.

5.7 Caching

Ollama client instance is cached for process lifetime. No artifact cache: reruns recompute everything. ImageSharp fonts are loaded per render call (minor cost).

6. API Documentation

This application **does not expose HTTP endpoints**. The internal API is `ProductionTools`. Agents see the `[Description]` strings. Below is the contract a developer or agent relies on.

6.1 Authentication

None on tools. ElevenLabs and optional OpenAI/Azure keys authenticate **outbound** calls only.

6.2 Tool catalog

Tool	Sync	Arguments	Return (string)	Errors
GetPipelineStatus	sync	none	multi-line status	none
LoadTranscript	sync	none	raw file text	FileNotFoundException
LoadNarrationSource	sync	none	rewritten talk	InvalidOperationException
RephraseTranscriptIntoNarration	async	none	"Saved N-word narration to {path}."	FileNotFound / InvalidOperation / HTTP timeout
SegmentTranscriptIntoEightScripts	sync	none	"Saved N narration scripts to {dir}."	count mismatch; empty transcript
SaveNarrationScripts	sync	segmentsJson	"Saved N agent-authored…"	JSON / count
PlanSlidesForAllSegments	sync	none	planned count + max	EnsureScripts

RenderAllSlideImages	sync	none	slides directory	EnsureScripts; IOException on PNG lock
SynthesizeAllAudioAsync	async	none	audio directory	EnsureScripts; ElevenLabs HTTP; ffmpeg
ComposeAllSegmentVideos	sync	none	video directory	missing audio/slides; ffmpeg
MergeAllVideos	sync	none	"Merged video: {path}"	need 8 videos; ffmpeg

`SaveNarrationScripts` payload: JSON array of objects with `number`, `title`, `script` (camelCase via source-gen). Extra slide/audio fields may deserialize but are unused until later tools fill them.

6.3 CLI "API"

```
NarrationStudio.exe [transcriptPath] [--orchestration handoff|sequential|direct]
NarrationStudio.exe --orchestration=direct
```

Unknown flags are ignored except they are not treated as transcript paths if they start with `--`. Confirmed: no `--help`.

6.4 Error responses

Tools throw .NET exceptions. MAF *inferred* surfaces those as tool errors to the model. Direct mode: `ProductionApp` logs `Production pipeline failed` and rethrows → process crash. No ProblemDetails JSON.

7. Frontend / Internal UI Workings

There is no component hierarchy, router, form library, or rendering lifecycle in the SPA sense.

7.1 Console as UI

- **Banner** in ProductionApp: transcript, output, mode, cleared file count, stub line, TTS provider.
- **Tool progress** in DirectPipelineRunner: each tool return string WriteLine.
- **Agent stream** in WorkflowHost: executor id headers and incremental `update.Update.Text`.
- **Host logging** Microsoft.Extensions.Logging to console (default Host logging).

7.2 State management

UI is stateless. PipelineStore is the application state. Operators infer progress from logs and from files appearing under `output/`.

7.3 Event handling

Ctrl+C triggers Host `ApplicationStopping`, which WorkflowHost passes into MAF streaming. Direct ffmpeg/TTS does not observe that token (limitation). Mutex is released when the process exits (using Dispose on Mutex).

7.4 Validation

No forms. Invalid CLI transcript path → FileNotFoundException before work starts.

8. Backend / Internal Service Workings

8.1 Business logic placement

Rule	Where enforced
Exactly 8 scripts	TranscriptSegmenter + ProductionTools count check + SaveNarrationScripts
Max 5 slides	SlidePlannerService clamp; RenderAllSlideImages Take(); options
Never segment raw transcript after rewrite exists	LoadNarrationSource / SegmentTranscriptIntoEightScripts
Overwrite on rerun	PipelineStore.Initialize delete + tools always write
Single instance	named Mutex
Spoken talk length ~7 min	prompt only (Ollama); stub file is used as-is

8.2 Controllers / handlers

`ProductionApp.RunAsync` is the only top-level handler. Tool methods are the command handlers.

8.3 Middleware

Agent middleware only: `ProductionGuardrails.RunAsync` (message filter) and `FunctionAsync` (tool log). Not ASP.NET middleware. Host logging is standard.

8.4 Background jobs / queues / events

None. No Hangfire, no Channel<T> pipeline, no bus. ffmpeg is synchronous from the caller's perspective (blocked WaitForExit). MAF streaming is async enumeration on the main run task.

8.5 External integrations

System	Used for	Failure mode
Ollama	Rewrite (if not stub) and default agent LLM	Timeout 30 min; TryCreate may still succeed while later calls hang/fail
OpenAI / Azure OpenAI	Optional agent LLM if keys/endpoint present	Takes priority over Ollama
ElevenLabs	TTS	Hard fail, no fallback
ffmpeg/ffprobe	MP3 encode, duration, slideshow, merge	Must be on PATH; 8 min timeout
Windows SAPI	TTS fallback if no ElevenLabs key	Falls back further to NAudio tone bed

9. Security

9.1 Authentication and authorization

No user login, roles, or tokens for using Narration Studio. Anyone who can run the EXE as the current user can produce videos and read `output/`.

Outbound: ElevenLabs API key; optional OpenAI key; optional Azure key or DefaultAzureCredential (managed identity / VS / az cli — *inferred* Azure.Identity behavior).

9.2 Data protection

Transcripts and generated media sit unencrypted on disk under `output/` (gitignored). Notes in pipeline-state.json may contain agent excerpts. PII redaction in guardrails is regex-only on agent messages, not on transcript files or TTS payloads. Emails in the source talk are sent to ElevenLabs in full. Confirmed: redaction does not wrap ProductionTools in direct mode.

9.3 Input validation

Filesystem paths are combined from config and args without a sandbox. A transcript argument of an unexpected file will be read if it exists. ffmpeg concat lists include absolute paths. There is no command-injection via ArgumentList (good: `ProcessStartInfo.ArgumentList` is used, not a single shell string).

9.4 Session / tokens

No app sessions. Named mutex is a lock, not auth. HttpClients do not implement retry/auth refresh.

9.5 Attack vectors and mitigations

Vector	Situation	Mitigation / gap
Prompt injection via transcript	Agent modes send status + files to an LLM	Guardrail blocked terms are trivial; tools still execute if the model calls them. Direct mode ignores transcript as instructions.
Secret leakage	API key in git-tracked appsettings.json	**Gap.** User requested appsettings storage. Do not commit to a public remote. UserSecretsId exists but is unused for ElevenLabs.
SSRF	Model:Endpoint is operator-controlled	Acceptable for a local tool; do not bind endpoint to untrusted input.
Path traversal	OutputDirectory from config	No jail. Operator-trusted config.
Resource exhaustion	Huge transcript → huge TTS bills / long ffmpeg	No size cap besides Ollama/stub length check of 400 min, not max.
Overlapping ffmpeg locks	two runs	Mutex

9.6 Secrets and configuration management

Confirmed sources: JSON, environment (Host defaults + explicit ElevenLabs env fallback), user-secrets package referenced but not loaded with a dedicated call (Generic Host **does** add user secrets in Development *inferred* when DOTNET_ENVIRONMENT=Development). Current launch profile does not set environment. ElevenLabs key in Studio section is the live source.

This document does not reprint secret values. Rotate any key that was committed or pasted into chat.

10. Error Handling and Edge Cases

10.1 Propagation

Exceptions from services → tools → DirectPipelineRunner or MAF tool invocation → ProductionApp catch logs Error and rethrows. No global exception handler / EventId taxonomy beyond default logger categories.

10.2 Logging

ILogger on ProductionApp, DirectPipelineRunner, AgentFactory, WorkflowHost, NarrationRewriteService, TtsService, ProductionGuardrails. Information for provider selection and ElevenLabs writes; Warning for stub rewrite, SAPI failure, agent create failure, incomplete agent merge, guardrail block; Error for pipeline failure.

No structured file log, no Application Insights, no correlation ID beyond RunId in state/filenames.

10.3 Retries

Operation	Retry
Delete generated files	Up to 8 attempts, backoff 150ms × attempt
PNG write	6 attempts + sidecar copy
ffmpeg merge	One fallback re-encode, not a counted retry
ElevenLabs / Ollama HTTP	None

10.4 Failure scenarios

- **Second instance:** InvalidOperationException mutex.
- **Missing transcript / stub:** FileNotFoundException.
- **Stub too short:** InvalidOperationException.
- **Player has video_*.mp4 open:** Initialize throws IOException after retries.
- **ffmpeg missing:** Win32Exception / "failed to start ffmpeg".
- **ElevenLabs 401/quota:** InvalidOperationException with body; run aborts mid-audio (earlier MP3s already written this run; next run wipes them).
- **Ollama down in handoff:** TryCreate may still construct a client; failure appears when the first model call happens. Direct mode with stubbing never needs Ollama.
- **Agent never merges:** failover DirectPipelineRunner.

10.5 Validation failures

Wrong agent JSON for SaveNarrationScripts, empty transcript, missing audio before compose. All throw; no partial merge if fewer than SegmentCount videos.

11. Configuration and Environment

11.1 Files

File	Role
`src/NarrationStudio/appsettings.json`	Primary Studio + Model settings; copied to output dir
`bin/Debug/net10.0-windows/appsettings.json`	What `dotnet run` actually binds when cwd is repo root
`appsettings.Development.json`	gitignored; optional if you add it

`Properties/launchSettings.json`	workingDirectory = repo root

11.2 Studio section (confirmed keys)

Key	Default in code	Meaning
SegmentCount	8	Scripts / MP3s / MP4s required
MaxSlidesPerSegment	5	Planner clamp and render trim
TargetTalkMinutes	7	Ollama word-budget; context instructions
UseStubNarration	true	Skip Ollama rewrite
StubNarrationPath	voice_transcript_ollama.txt	Relative stub file
TranscriptPath	empty	Optional explicit transcript
OutputDirectory	output	Relative to cwd unless rooted
OpenAIModel	gpt-4o-mini	If OPENAI_API_KEY set
AzureOpenAIEndpoint / Deployment	empty / gpt-4o-mini	Agent client if endpoint set
ElevenLabsApiKey	empty in class; set in JSON	TTS
ElevenLabsVoiceId	UPXQvvSDDubCtocmbltQ	TTS voice
ElevenLabsModelId	eleven_multilingual_v2	TTS model
VideoWidth / Height / FrameRate	1920 / 1080 / 30	PNG and ffmpeg vf

11.3 Model section

Endpoint default http://localhost:11434; Model qwen3.8:latest; TimeoutMinutes 30.

11.4 Environment variables

Variable	Effect
ELEVENLABS_API_KEY	Used only if JSON key is empty
OPENAI_API_KEY / OPENAI_MODEL	Agent chat client
AZURE_OPENAI_ENDPOINT / API_KEY / DEPLOYMENT_NAME / DEPLOYMENT	Azure agent client; endpoint wins over OpenAI and Ollama

DOTNET_ENVIRONMENT	Standard Host; not set by launch profile

11.5 Feature flags

The only first-class flag is `UseStubNarration`. Orchestration mode is a CLI flag, not JSON. No staging/production splits exist in-repo.

12. Deployment and Infrastructure

Confirmed: this is a local developer executable, not a deployed service.

- **Build:** `dotnet build` / Visual Studio 2026 on `NarrationStudio.slnx`. SDK-style csproj, TargetFramework net10.0-windows.
- **Run:** `dotnet run --project src\NarrationStudio` with cwd repo root (launchSettings) or F5 in VS.
- **Hosting:** operator's Windows 10/11 machine. Requires ffmpeg/ffprobe on PATH, optional Ollama, optional network to ElevenLabs.
- **Containers / k8s / CI/CD / monitoring:** none in the repository. No Dockerfile, no GitHub Actions, no health checks.
- **Publish:** `dotnet publish` would produce a framework-dependent or self-contained exe; not scripted here. System.Speech and Windows TFM mean Linux containers are not a supported target without replacing SAPI.

Observability is console logs only. Operators watch `output/` for artifacts.

13. Performance and Scalability

Stage	Cost	Notes
Stub rewrite	Negligible	File copy + strip
Ollama rewrite	Minutes; up to 30 min timeout	Single request; temperature 0.35
Segment / plan	Milliseconds	Regex
PNG render	Seconds for ~40 images	Sequential CPU; ImageSharp
ElevenLabs TTS	Dominant when enabled	8 sequential HTTP calls, 3 min timeout each worst case
ffmpeg compose	High CPU	8 sequential encodes 1920×1080
ffmpeg merge copy	Fast	Re-encode fallback is slow

Concurrency. No parallel TTS or ffmpeg. Could be added but would contend on API rate limits and disk. Mutex forbids two pipelines.

Scalability. Designed for one talk, one machine, one run. Not multi-tenant. Bottleneck for "correct sounding" video is ElevenLabs quota/latency; bottleneck for local-only is SAPI + ffmpeg.

Caching. Intentionally disabled for media so reruns match current narration.

14. Testing

Confirmed: the repository contains no test project, no xUnit/NUnit/MSTest references, and no sample fixtures beyond the two transcript text files.

What a new developer should still treat as the implicit test matrix (manual):

- direct + stub + ElevenLabs → eight MP3s + merged mp4
- direct + stub + no API key → SAPI path
- UseStubNarration false with Ollama up → narration-rewritten.txt from model
- handoff failover when merge missing
- second instance mutex
- rerun deletes previous video_*.mp4 and regenerates from stub
- locked output file → clear failure message

Highest-value future unit tests: TranscriptSegmenter distribution count, SlidePlanner max-5, StripModelExtras, PipelineStore wipe, TtsService key resolution (without calling network).

15. Developer Guide

15.1 How to understand the codebase

Read in this order: `Program.cs` → `ProductionApp.cs` → `DirectPipelineRunner.cs` → `ProductionTools.cs` → the service for the stage you care about → only then `AgentFactory` / `WorkflowHost`. Agents are a control plane over tools; tools are the product.

15.2 Where functionality lives

If you want to change…	Start here
CLI, mutex, mode default	ProductionApp
File names / wipe policy	PipelineStore
Stub vs Ollama rewrite	NarrationRewriteService, StudioOptions

How text is chopped into 8	TranscriptSegmenter
On-screen phrases	SlidePlannerService
Slide look	SlideImageRenderer
Voice / TTS vendor	TtsService, appsettings
Video timing / codecs	VideoComposerService
Agent personalities / tool grants	AgentFactory
Handoff graph	WorkflowHost
Blocked words	ProductionGuardrails

15.3 Extending

- **Ninth segment:** raise SegmentCount; verify ffmpeg merge and agent instructions (several prompts hardcode “8”).
- **New specialist:** add tool method, wrap in AgentFactory, add handoff edges, update sequential list and coordinator text.
- **Cloud LLM rewrite:** swap OllamaChatClientFactory usage in NarrationRewriteService; keep stub flag.
- **Stable output name:** today merged name includes RunId; consumers must read the Done line or pipeline-state.json.

15.4 Common workflows

```
cd E:\exercises\maf
dotnet build
dotnet run --project src\NarrationStudio -- --orchestration direct
```

Edit transcripts at repo root. Inspect `output/scripts/narration-rewritten.txt` to verify stub/Ollama. Play the newest `output/video_*.mp4` .

15.5 Pitfalls and technical debt

- appsettings loaded from bin copy; editing source JSON requires rebuild/copy.
- Secrets in tracked JSON.
- Agent prompts still mention “8” and audio-01..08 even if SegmentCount changes.
- Guardrail slide counter does not enforce.
- CancellationToken.None on TTS/rewrite.
- SummarizeForAgent next-step order prefers audio before remaining slide plans if scripts exist — odd if someone called TTS without planning (direct mode does not do that).

- No tests.
- Tone-bed fallback can produce a video that “works” with a beep instead of speech if SAPI fails and no ElevenLabs key.
- FileMemoryProvider types come from MAF; upgrading Microsoft.Agents.AI can break AgentFactory.

16. Complete Dependency Map

16.1 Internal

```
Program
  └─ ProductionApp
        ├─ StudioOptions, TtsService (banner)
        ├─ PipelineStore
        ├─ ProductionTools ──┬─ NarrationRewriteService — OllamaChatClientFactory
        │                    ├─ TranscriptSegmenter
        │                    ├─ SlidePlannerService
        │                    ├─ SlideImageRenderer
        │                    ├─ TtsService — FfmpegProcess, NAudio, System.Speech, HttpClient
        │                    └─ VideoComposerService — FfmpegProcess
        ├─ AgentFactory ──────── ProductionTools, PipelineStore, ProductionContextProvider,
        │                        OllamaChatClientFactory, ProductionGuardrails (new per TryCreate),
        │                        FileMemoryProvider (package)
        ├─ WorkflowHost ──────── PipelineStore, ProductionCrew
        └─ DirectPipelineRunner — ProductionTools, PipelineStore
```

16.2 NuGet packages (csproj) and why

Package	Why	Where
Microsoft.Extensions.Hosting 10.0.0	Generic host, DI, logging, config	Program.cs
Microsoft.Extensions.Configuration.*	JSON, env, user secrets	Program.cs, csproj
Microsoft.Agents.AI 1.17.0	AIAgent, AsAIAgent, context, file memory types	AgentFactory, Guardrails, Memory
Microsoft.Agents.AI.Workflows 1.17.0	Handoff/sequential workflows, InProcessExecution	WorkflowHost
Microsoft.Agents.AI.OpenAI 1.17.0	OpenAI/Azure IChatClient adapters	AgentFactory
Microsoft.Extensions.AI 10.7.0	IChatClient, ChatMessage, AIFunctionFactory	Agents, rewrite, tools wrap
Microsoft.Extensions.AI.OpenAI 10.6.0	AsIChatClient extensions	AgentFactory
Azure.AI.OpenAI 2.1.0 / Azure.Identity 1.14.2	Azure client + DefaultAzureCredential	AgentFactory

OpenAI (transitive via packages)	OpenAIClient	AgentFactory
OllamaSharp 5.4.25	OllamaApiClient	OllamaChatClientFactory
SixLabors.ImageSharp 3.1.11 + Drawing + Fonts	PNG slides	SlideImageRenderer
NAudio 2.2.1	Tone-bed fallback WAV	TtsService.GenerateToneBed
System.Speech 9.0.8	Windows TTS	TtsService

NoWarn includes MAAI001, OPENAI001, MEAI001 — experimental / preview analyzer IDs for MAF/OpenAI/MEAI.

16.3 External runtime dependencies

ffmpeg, ffprobe, Windows, optional Ollama daemon, optional ElevenLabs network, optional Azure/OpenAI network.

17. Technical Diagrams

17.1 Overall architecture

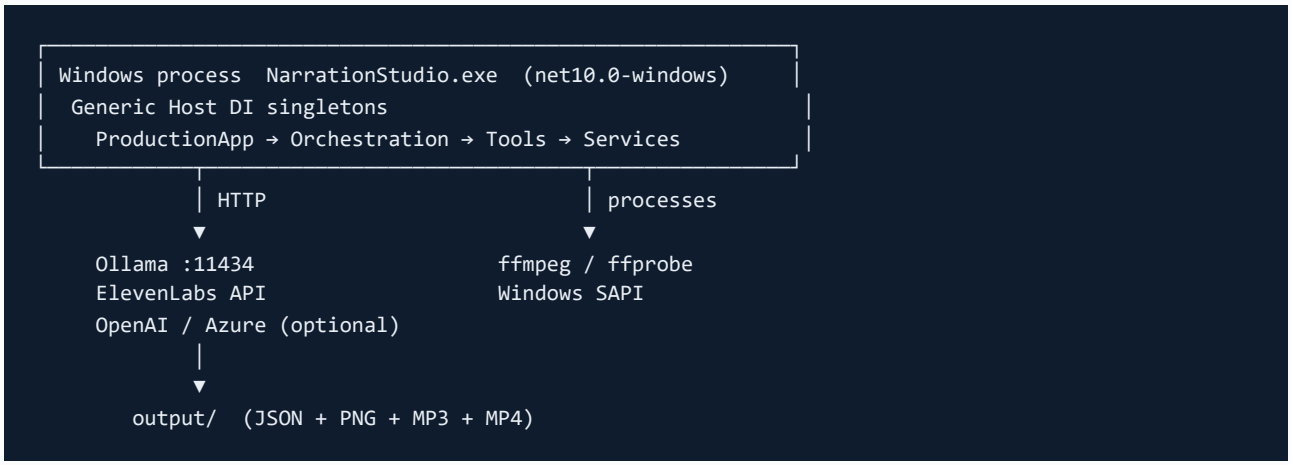


Figure 2. Process and externals. Confirmed.

17.2 Component relationships

See §16.1. All arrows are constructor injection except ffmpeg (Process.Start) and HTTP clients created inside services.

17.3 Data flow

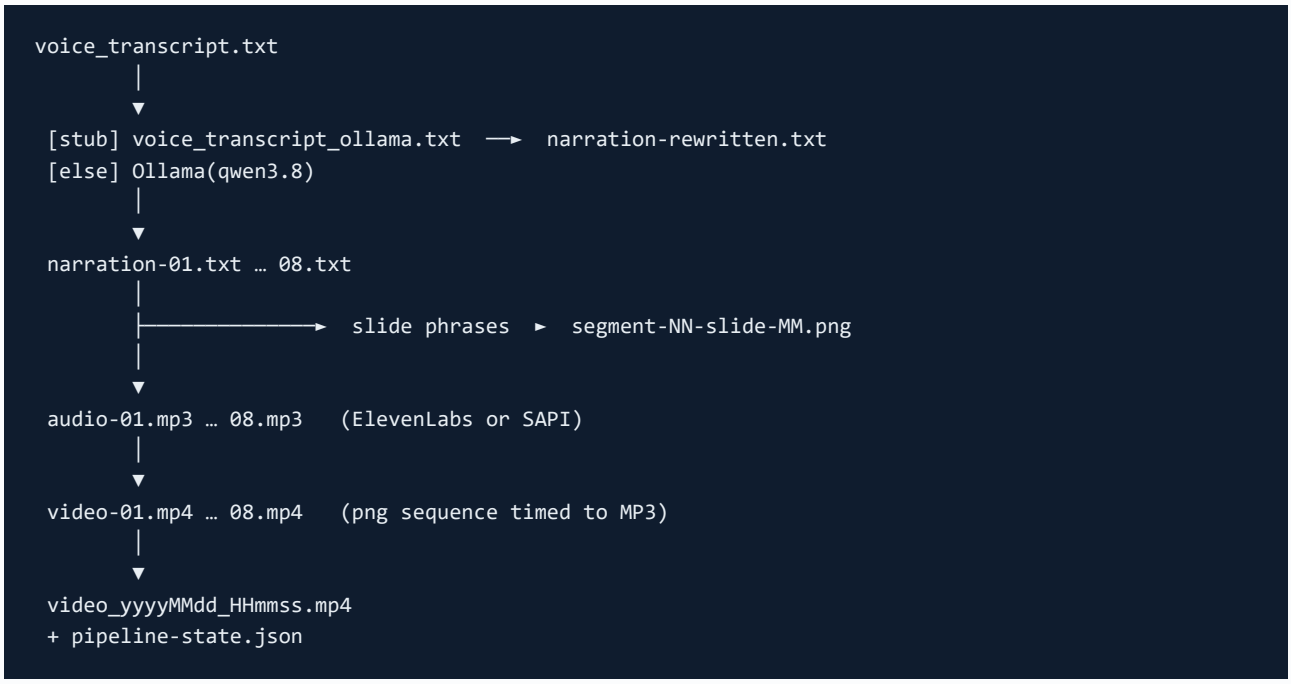


Figure 3. Artifact lineage. Later stages must not read the raw transcript once narration-rewritten.txt exists.

17.4 Direct-mode lifecycle

See §4.1. Sequence is strictly linear; each tool Mutate persists JSON before the next tool starts (except render which mutates once after all PNGs).

17.5 Handoff lifecycle

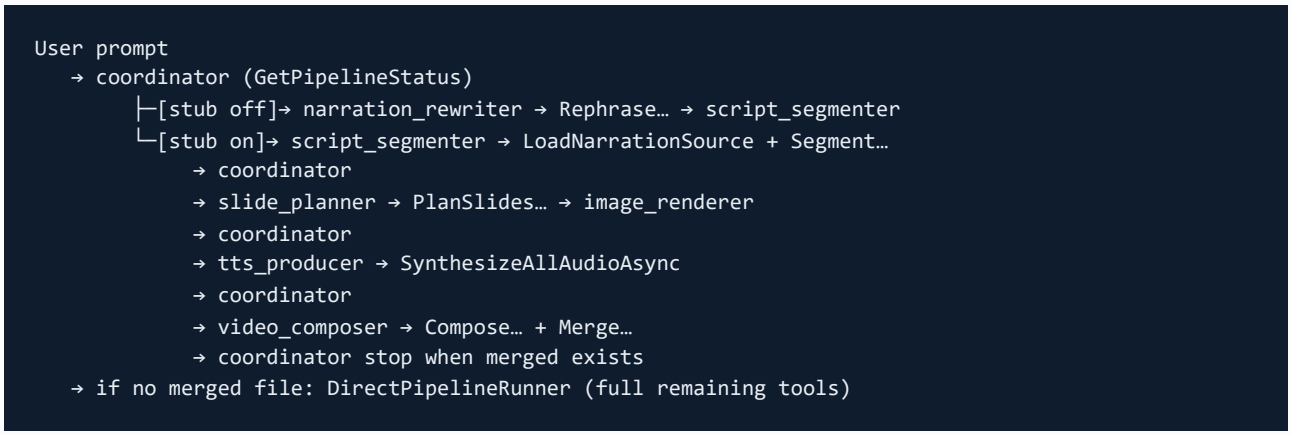


Figure 4. Handoff control. Routing is model-directed; tools perform I/O.

17.6 Data “ER” diagram

```
PipelineState
   runId PK (string, time-based)
   1 ──< NarrationSegment.number  (1..SegmentCount)
              1 ──< SlideSpec.number (1..≤5)
   notes[]  (capped 24)
```

Figure 5. In-memory / JSON relationship. No RDBMS.

17.7 Authentication flow

```
Operator runs EXE as Windows user
   → no login
   → TtsService.ResolveApiKey: JSON then env
        → if set: xi-api-key on ElevenLabs POST
   → AgentFactory.CreateChatClient:
        Azure endpoint? → ApiKeyCredential or DefaultAzureCredential
        else OPENAI_API_KEY? → OpenAIClient
        else Ollama (no auth in this repo)
```

Figure 6. Outbound credential flow only.

17.8 Deployment architecture

Single box: developer workstation. No load balancer, no container, no CDN. Distribution, if any, is copying the published folder plus ensuring ffmpeg and appsettings are present.

18. Appendix

18.1 Glossary

Term	Meaning in this codebase
MAF	Microsoft Agent Framework (NuGet Microsoft.Agents.AI*)
Direct mode	No agents; DirectPipelineRunner calls tools in order
Handoff	Coordinator routes to specialist agents
Stub narration	Pre-written talk in voice_transcript_ollama.txt used instead of Ollama rewrite
RunId	Local timestamp used in merged filename

ProductionTools	The in-process API agents and direct mode share
Guardrails	Agent middleware for blocked terms and PII regex; not a full policy engine
SAPI	Windows Speech API used when no ElevenLabs key

18.2 Important files and directories

```
E:\exercises\maf\
  NarrationStudio.slnx
  README.md
  voice_transcript.txt              raw source notes
  voice_transcript_ollama.txt       stub 7-minute talk
  output\                           gitignored generated tree
  src\NarrationStudio\
    Program.cs
    ProductionApp.cs
    appsettings.json
    NarrationStudio.csproj
    Agents\AgentFactory.cs
    Agents\WorkflowHost.cs
    Orchestration\DirectPipelineRunner.cs
    Tools\ProductionTools.cs
    Services\*.cs
    Models\PipelineModels.cs
    Configuration\*.cs
    Guardrails\ProductionGuardrails.cs
    Memory\ProductionContextProvider.cs
    Properties\launchSettings.json
  docs\                             this document (HTML + PDF)
```

18.3 Configuration reference

See §11. Do not copy secrets into documentation, tickets, or screenshots.

18.4 Key assumptions

- Operator is on Windows with .NET 10 SDK and ffmpeg on PATH.
- One talk at a time; output directory is expendable.
- Transcript language works with the chosen TTS model/voice.
- Default run uses stub narration and (if configured) ElevenLabs.
- Repo root is the working directory so relative output/ and transcripts resolve.

18.5 Known limitations

- No GUI, API server, DB, tests, CI, or multi-run resume.
- Agent quality depends on the LLM; direct mode is the reliable factory.

- Hardcoded “eight” in several prompts.
- Secrets currently live in appsettings.json.
- No max transcript size; ElevenLabs character limits can fail a segment.
- Slide timing is even split of audio duration, not word-level alignment.
- Guardrails do not apply to direct mode and do not enforce slide counts.

18.6 Areas requiring further investigation

- Exact MAF handoff / autonomous turn semantics on package 1.17.0 (only usage sites are in WorkflowHost).
- FileMemoryProvider on-disk layout (deleted every run; low priority).
- Whether Host user-secrets load without Development environment.
- ElevenLabs voice settings (stability/similarity) — currently only text + model_id.
- ffprobe duration locale parsing (uses current-culture TryParse, not InvariantCulture) — possible issue on some locales.

End of document. Generated from the Narration Studio source tree as of 21 August 2026. If the code and this PDF diverge, the code wins.